# Common setup (run once)

```
pip install nltk
```

```
import nltk

# Download the NLTK datasets/models needed by the lab programs.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('brown')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('universal_tagset')
nltk.download('conll2000')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

Run each program in Jupyter Notebook, Google Colab, or VS Code with Python installed. Some programs download corpora on first run and need an internet connection.

# 9. Calculate emission and transition matrices for POS tagging using HMM

**Aim:** To calculate the transition and emission probabilities required by a Hidden Markov Model (HMM) POS tagger.

• **Transition probability:** Probability of a POS tag given the preceding tag.

• **Emission probability:** Probability of a word given its POS tag.

\[

P(t_i\mid t_{i-1})=\frac{C(t_{i-1},t_i)}{C(t_{i-1})}

\]

\[

P(w_i\mid t_i)=\frac{C(t_i,w_i)}{C(t_i)}

\]

```
import nltk
from nltk.corpus import brown
from collections import Counter, defaultdict

# Download the Brown Corpus and universal tagset.
nltk.download('brown')
nltk.download('universal_tagset')

# Obtain a small tagged corpus.
# Each item contains (word, POS tag).
tagged_sentences = brown.tagged_sents(
    categories='news',
    tagset='universal'
)[:100]

# Store counts needed for the HMM probabilities.
transition_counts = Counter()
tag_counts = Counter()
emission_counts = Counter()
word_tag_counts = Counter()

for sentence in tagged_sentences:
    previous_tag = '<START>'

    for word, tag in sentence:
        word = word.lower()

        # Count transitions between consecutive tags.
        transition_counts[(previous_tag, tag)] += 1

        # Count how often each tag occurs.
        tag_counts[tag] += 1
```

```
        # Count how often a tag emits a word.
        emission_counts[(tag, word)] += 1

        # Count the number of times each word occurs.
        word_tag_counts[word] += 1

        previous_tag = tag

    # Count the transition from the final tag to sentence end.
    transition_counts[(previous_tag, '<END>')] += 1

# Calculate transition probabilities.
print("Transition probabilities:")

for (previous_tag, current_tag), count in transition_counts.most_common(10):
    probability = count / tag_counts[previous_tag] if previous_tag != '<START>' else (
        count / len(tagged_sentences)
    )

    print(previous_tag, "->", current_tag, ":", round(probability, 4))

# Calculate emission probabilities.
print("\nEmission probabilities:")

for (tag, word), count in emission_counts.most_common(10):
    probability = count / tag_counts[tag]
    print(tag, "emits", word, ":", round(probability, 4))
```

## Sample output

```
Transition probabilities:
NOUN -> NOUN : [calculated probability]
DET -> NOUN : [calculated probability]
...

Emission probabilities:
NOUN emits company : [calculated probability]
VERB emits said : [calculated probability]
...
```

The exact values depend on the Brown Corpus version and the number of sentences selected.

**Explanation:** The transition model describes tag sequences, while the emission model describes how likely a word is under a given tag. Together, they are used to determine the most likely POS-tag sequence.